

ML System Design Document

Track: Machine Learning Engineering - HealthConnect No-Show Prediction

1. ML Problem Overview

HealthConnect Clinic loses operational efficiency and patient-care capacity when scheduled patients fail to attend appointments (“no-shows”): slots go unused, staff time is wasted, and other patients are denied earlier access to care. The goal of this workstream is to design a reliable, reproducible machine learning system that scores upcoming appointments by their likelihood of resulting in a no-show, early enough for staff to intervene (e.g. extra reminder, phone confirmation, controlled overbooking, waitlist backfill).

Scope decision on appointment outcomes: the dataset's target column, `appointment_outcome`, has three values ,Attended (46.3%), No-Show (48.5%), and Cancelled (5.3%). For the initial (v1) system, the ML problem is scoped as binary classification: No-Show vs Attended. Cancelled appointments are excluded from model training and evaluation, because a cancellation is a distinct, patient-initiated event that is already known to staff in advance (the slot can be reallocated immediately) it is not the same phenomenon as a silent no-show, and folding it into either class would blur the label's meaning. This is a scoping decision, not a data-quality fix; a future iteration could extend the system to a 3-class problem or a separate cancellation-risk model if the business finds that valuable.

2. Input and Output Definition

Output

A probability score (0–1) that a given upcoming appointment will be a No-Show, produced far enough ahead of the appointment date to be actionable (e.g. computed at booking time and refreshed after the reminder step), plus a binary risk flag at a threshold to be tuned once real evaluation data is available.

Candidate Input Features (available before the appointment occurs)

Category	Fields	Notes
Demographics	age, age_group, gender	age_group is derivable from age (redundant, kept for interpretability)
Appointment details	appointment_type, appointment_day, appointment_time, booking_lead_days	appointment_day is derivable from appointment_date; booking_lead_days derivable from booking_date/appointment_date , both confirmed internally consistent
Patient history	previous_appointments, previous_no_shows (+ a derived historical no-show rate)	Strongest observed signal: no-show rate rises from 43.5% (0 prior no-shows) to 68.8% (3+ prior no-shows)
Reminder	reminder_sent, reminder_channel	reminder_channel is missing for exactly the rows where reminder_sent = No (1,366 rows) logical, not a data quality defect;

		must be encoded as its own “not applicable” category, not imputed
Logistics	distance_to_clinic_km	Known at booking time from patient address; legitimate input

Excluded from Input: waiting_time_minutes

This field is flagged as a data leakage / validity risk and excluded from v1 modelling. Data inspection shows waiting_time_minutes is populated even for No-Show (2,386 of 2,423 rows) and Cancelled (261 of 263 rows) appointments which is logically questionable, since a patient who never arrived should not have a measurable clinic waiting time. This suggests the field is either (a) simulated/placeholder data not tied to actual attendance, or (b) only knowable after or during the appointment in a real system. Either interpretation makes it unsafe to use as a pre-appointment predictive input: including it would risk data leakage and would not be available at the moment a real production system needs to make its prediction. This should be raised with the data/analytics team as a clarifying question before Week 5 modelling begins.

Identifiers (appointment_id, patient_id) are not used as direct predictive features, but patient_id is needed to correctly compute per-patient historical aggregates (previous_appointments, previous_no_shows) without leaking information across the train/test time boundary, see Section 4.

3. High-Level System Architecture

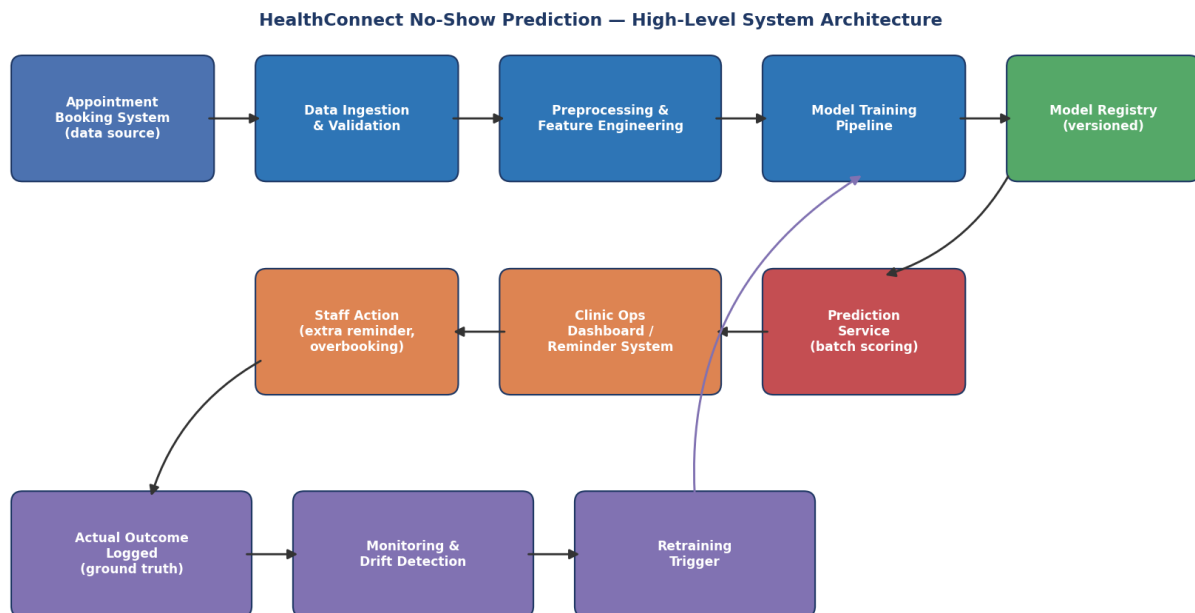


Figure 1. Proposed high-level architecture: ingestion → feature engineering → training → registry → batch prediction → clinic action → outcome logging → monitoring/retraining.

The design favors a batch scoring pattern over a real-time API for v1: appointments are typically booked days or weeks in advance (booking_lead_days ranges 0–60 in the sample data), so there is no operational need for millisecond-latency inference. A scheduled batch job (e.g. nightly) that scores all upcoming appointments and

pushes high-risk flags into the clinic's reminder/ops workflow is simpler to build, test, and monitor than a real-time service, and matches the clinic's actual decision cadence.

4. Proposed ML Workflow

- 1. Data Ingestion & Validation : load the appointment export, validate schema against the Data Dictionary, run the consistency checks already confirmed in Week 4 (booking_lead_days, appointment_day, previous_no_shows $\leq$ previous_appointments) on every new batch.
- 2. Data Cleaning & Feature Engineering : encode reminder_channel's missingness as a category rather than imputing; decide an imputation strategy for the genuinely missing distance_to_clinic_km (90 rows) and any residual gaps; engineer a per-patient historical no-show rate feature computed only from appointments strictly before the one being scored.
- 3. Temporal Train/Test Split : because appointment dates in the sample span January 2025 to June 2026, splitting chronologically (train on earlier appointments, test on later ones) rather than randomly better simulates real production conditions, where the model always predicts the future from the past. A random split risks leaking future patterns into training and overstating offline performance.
- 4. Model Training & Evaluation : baseline (Logistic Regression) plus at least one ensemble model, evaluated primarily on Recall/ROC-AUC given the operational cost of missing a likely no-show (an empty slot) versus a false alarm (one extra reminder).
- 5. Model Registry & Versioning : every trained model artifact is saved with a version identifier, training data snapshot reference, and evaluation metrics attached, so any prediction can be traced back to the exact model and data that produced it.
- 6. Batch Inference : scheduled job scores upcoming appointments and writes risk flags to a location the clinic ops/reminder workflow can consume.
- 7. Integration : high-risk flags feed into staff action (extra reminder via the preferred channel, phone confirmation, or controlled overbooking).
- 8. Monitoring & Retraining : log actual outcomes as they occur, track prediction accuracy and feature distributions over time (drift detection), and trigger retraining on a schedule or when drift/performance thresholds are breached.

5. Key Dependencies

Type	Dependency
Data	Continued access to the appointment export (or, ideally, a direct feed from the booking system); Data Dictionary kept in sync with any schema changes
Data quality	Clarification needed from the data/analytics team on the waiting_time_minutes anomaly before it can be safely ruled in or out of any feature set
Model	scikit-learn (or equivalent) for training; consistent feature-engineering code shared between training and inference to avoid training-serving skew
System	A destination for batch predictions , the clinic's reminder system or ops dashboard for the output to be actionable, not just descriptive
Team / process	Alignment with the Data Analytics and Data Science tracks working on the same HealthConnect dataset in parallel this week, to avoid divergent target-variable or feature definitions

Infrastructure	Git/GitHub for code versioning; a simple model registry/versioning convention (e.g. model_v{n}_{date}.pkl plus a metrics log) in the absence of dedicated MLOps tooling at this stage
----------------	---

6. Reproducibility, Versioning & Model Management

- Fix random seeds in all data splitting and model training code.
- Never overwrite the original HealthConnect_Appointment_Data.csv; all cleaned/derived datasets are saved under new, clearly named files (per the project's resource-handling instructions).
- Pin library versions in a requirements.txt so the environment is reproducible.
- Version model artifacts explicitly (filename or registry entry including date/version) alongside the metrics and data snapshot used to produce them.
- Document every preprocessing and feature-engineering decision inline (notebook markdown or README), especially the exclusions and scoping decisions made in this document (Cancelled exclusion, waiting_time_minutes exclusion).

7. Initial Implementation Approach

Week 4 deliverable is this design document only — no code or model is built yet, per the assignment's scope. The proposed sequencing for subsequent weeks:

- Week 5: build the reproducible ingestion + validation + feature engineering pipeline; implement the temporal train/test split; produce a first baseline model with a documented evaluation.
- Week 6+: iterate on model choice and features, wrap the model behind a simple, versioned prediction function; add basic monitoring/logging hooks; begin drafting the integration point with the clinic's reminder workflow (even if simulated at first, given this is a fictional clinic).

8. Assumptions, Limitations, Risks and Dependencies

Category	Item
Data limitation	waiting_time_minutes shows values for the large majority of No-Show and Cancelled rows, which is logically inconsistent with its stated meaning treated as a data quality/leakage risk and excluded pending clarification.
Missing data	distance_to_clinic_km (90 rows, 1.8%) and waiting_time_minutes (60 rows, 1.2%) have missingness spread across all outcome categories, consistent with random/technical gaps rather than a logical pattern requires an explicit imputation decision before modelling.
Not a defect	reminder_channel is missing for exactly the 1,366 rows where reminder_sent = No , this is expected and must be encoded as a category, not treated as missing data to impute.
Modelling consideration	Class balance is close to even once Cancelled rows are excluded (~48.5% No-Show vs ~46.3% Attended of the original data), so heavy resampling is likely unnecessary, but should be re-verified on the binary subset in Week 5.
Ethical / fairness concern	age and gender are sensitive attributes. If used as direct model inputs, the resulting risk scores must be audited for disparate impact across age and gender groups before any

	operational use (e.g. before letting the score influence overbooking or reminder prioritization).
Technical constraint	The dataset is fictional/synthetic; real-world deployment would require re-validation against genuine clinic behavior and monitoring for concept drift (e.g. seasonal patterns, policy changes) not present in a static synthetic sample.
Project risk	Parallel problem-framing work by the Data Analytics and Data Science tracks on the same dataset could diverge from this document's scoping decisions (e.g. handling of Cancelled, feature list) without a short alignment check-in.
System dependency	No existing production ML infrastructure is assumed at HealthConnect Clinic; the batch-scoring design in Section 3 is chosen specifically because it needs minimal new infrastructure.